

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

First Semester of M. Tech. Examination (CE)

Dec 2012

CE705 Advanced Compiler Construction

Date: 22.12.2012, Saturday

Time: 10:00 a.m. To 01:00 p.m.

Maximum Marks: 70

Instructions:

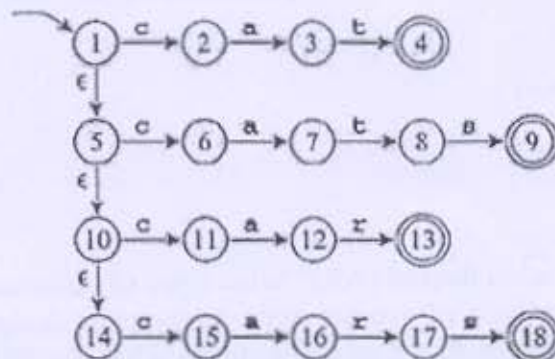
1. All questions are compulsory.
2. Indicate clearly, the option you attempt along with its respective question number.
3. Figures to right indicate marks.
4. Rough work is done in the last page of main answer sheet; don't write anything on the question paper.

SECTION - I

- Q - 1 (a) "Interpreters are slow compare to compilers in execution". Justify the statement. [02]
(b) Explain the need of 2 pass compiler for the modern programming language. [02]
(c) Is it possible to use grammar instead of regular expression in lexical analysis? Justify your answer. [03]
- Q - 2 (a) For $(a | b)^*$ construct the syntax tree and find $nullable(n)$, $firstpos(n)$, $lastpos(n)$, and $followpos(p)$. [04]
(b) Consider the following regular expression [07]
 $(0 | \epsilon)(0 | 1)^*0$
i) Construct an NFA for the regular expression above using Thomson's construction.
ii) Convert the NFA to a DFA using the subset construction algorithm.
- (c) What is input buffering in lexical analysis? Explain with example. [03]

OR

- Q - 2 (a) Write a Lex program that copies a C program, replacing each instance of the keyword float by double. [04]
(b) Convert following NFAs to deterministic finite automata. Using the algorithm for state minimization construct a minimized DFA. [07]



- (c) What are the advantages to a language-processing system in which the compiler produces assembly language rather than machine language? [03]

- Q - 3 (a) Construct canonical LR (1) item sets for the following grammar. [06]
 $S \rightarrow SB \mid Ca$
 $B \rightarrow Bb \mid \epsilon$
 $C \rightarrow aB \mid c$
- (b) Construct operator precedence parsing table for following grammar. [04]
 $S \rightarrow (L) \mid a$
 $L \rightarrow L, S \mid S$
- (c) With suitable example explain how shift-reduce and reduce-reduce conflict can be removed from parsing table. [04]

OR

- Q - 3 (a) Construct LALR (1) item sets for the following grammar. [06]
 $D \rightarrow L : T$
 $L \rightarrow L, id \mid id$
 $T \rightarrow int$
- (b) Design grammars for the following languages: [04]
 1. The set of all strings of 0s and 1s with an unequal number of 0s and 1s.
 2. The set of all strings of 0s and 1s in which 011 does not appear as a substring.
- (c) Show that the following grammar: [03]
 $S \rightarrow SA \mid A$
 $A \rightarrow a$
 is SLR(1) but not LL(1).

SECTION - II

- Q - 4 (a) What is intermediate code generation? Explain its usage with example. [04]
 (b) Convert the following code in to the (Extended/Reverse) Polish Notation. [03]

```

j=1;
i=0;
while(i<10)
{
    j=j*i;
    i=i+1;
}
  
```
- Q - 5 (a) Which kind of optimization can be performed on the following code? Show optimized code also. [03]

```

i=10;
for(int j=0; i<j; j++)
{
    i=8000;
    b[j]=i;
}
  
```
- (b) What is the Activation Record (AR)? What types of information are kept in AR? [04]
 (c) Discuss the importance of symbol table in compiler design. How is the symbol table manipulated at various stages of a compiler? What should be the typical entries in the symbol table? [07]

OR

- (c) Show the typical sub division of run-time memory into code and data (Memory Map). [07]
 How the information needed by a single execution of a procedure is managed?

Q - 6 (a) Draw the Syntax Directed Definition that verifies the parenthesis are balanced or not in given input string. Indicate inherited attributes and synthesized attributes in the tree. [06]

OR

Q - 6 (a) For input expression $(5*8+1)*4$, construct an annotated parse tree according to the syntax directed translation. Write the difference between inherited attribute and synthesized attribute. [06]

(b) What is the necessity of optimization in compilation? Can human being optimize a program better than compiler? Justify your answer. [04]

(c) Explain the issues in generating target code with suitable example. [04]